Self-Crystallizing Behavior Trees: Amortizing Foundation-Model Reasoning into Deterministic Control for Autonomous Drones

Anonymous Author(s)

***Abstract*—Classical robot control architectures, Finite State Machines (FSMs) and Behavior Trees (BTs), are deterministic, inspectable, and fast, but fixed at design time: they fail on the long tail of situations their designers did not anticipate. Foundation models can synthesize new control logic at runtime, at the price of latency, cost, and non-determinism that are hard to accept in safety-critical aerial systems. Existing LLM-driven approaches keep the model permanently in the loop and discard what it produces after each use. We propose *Self-Crystallizing Behavior Trees* (SCBT), in which the foundation model is a *compiler* rather than a *runtime*. When a node of a vehicle's BT fails, an LLM proposes a minimal subtree *patch* in a constrained, mission-agnostic grammar; CMA-ES tunes the patch's continuous parameters in simulation; a contextual bandit decides whether a stored patch or a new LLM call is worth paying for; and a three-stage admission pipeline (static check, invariant monitors over randomized replays, and an empirical margin over the unpatched tree) admits only patches that demonstrably recover. Admitted patches are stored in a fleet library indexed by failure context and, once the LLM call rate has decayed, *crystallized* into a conventional deterministic BT that no human wrote. We implement SCBT on Microsoft Project AirSim with a schema-constrained LLM pipeline (Google ADK, Gemini 3.1 Pro) and a lockstep simulator. Over three seeds of a 20-episode learning phase with injected wind-gust and battery failures, SCBT used 11 ± 1.4 LLM calls against 26 for whole-tree regeneration and 22 for patching without a library, with 0.33 calls per episode in the second half. The crystallized trees, with 28 and 33 nodes and two patches each, reached $97 \pm 2\%$ mission success on the curriculum with no learning component running, against 12% for the unpatched base tree and 98% for a hand-coded expert tree; re-ticked offline against recorded observations they reproduce their online decision traces exactly. Two negative results are reported: replay admission as parameterized rejected useful wind recoveries because it attributed wind-induced geofence excursions to the patch, and the static-only ablation generalized better to held-out failures as a consequence.**

***Index Terms*—behavior trees, large language models, contextual bandits, CMA-ES, UAV, neuro-symbolic AI, lifelong learning, simulation-based verification**

I. Introduction

For half a century robot behavior has been specified explicitly, first as FSMs and, over the last decade, as BTs [1], [2]. These architectures are attractive precisely because they are *closed*: every branch can be inspected, every transition is deterministic, and the controller runs at a fixed rate on embedded hardware. The same closedness is their limitation. A BT cannot execute a branch that was never written, and in open environments the set of situations that require a branch is effectively unbounded. We call this *design-time rigidity*.

Foundation models offer an apparent escape: LLMs write robot programs from natural-language goals [9], generate BT XML on demand [15], [16], ground plans in perceptual affordances [10], [12], and coordinate heterogeneous teams [26], [27]. But these systems share a structural assumption: the model stays in the control loop for the lifetime of the system, and what it produces is discarded after use. For aerial robots this has three consequences. **Latency**: in our implementation a single structured proposal from a frontier model takes 31–105 s (two schema-constrained calls); a vehicle in a wind gust cannot wait for it. **Cost**: operating cost grows with activity. **Non-determinism**: two flights under identical conditions may execute different logic, which defeats the certification argument that makes BTs acceptable in the first place.

The two camps are usually framed as a trade-off between adaptability and determinism. We argue the trade-off is an artifact of *when* the foundation model is invoked. If the model is called only when the existing control structure fails, if what it produces is verified and *retained*, and if a learned policy stops calling it once stored structure suffices, then adaptability is paid for once, during a learning phase, and the deployed system is an ordinary deterministic BT.

A. Approach

SCBT assigns each mechanism to the job it is suited for:

- The **LLM proposes structure**: given a failed node and its context, it emits one minimal, local, mission-agnostic patch in a constrained grammar. Structure synthesis is discrete, symbolic, and rare.
- **CMA-ES tunes parameters**: each patch declares its uncertain numeric ports as tunables with bounds; these are optimized against simulated replays before admission.
- A **contextual bandit selects**: among stored patches that match the failure context and the option of a new LLM call, with the call's cost in the reward.
- A **three-stage admission pipeline** gates library entry: static check against the grammar and the vehicle's capability manifest; invariant monitors over randomized replays; and an empirical success margin over the tree as it currently stands.
- A **deterministic BT executes**: the vehicle controller is always a plain BT ticked in lockstep with the simulator. No neural component is in the tick path.

B. Contributions

1) The SCBT framework: failure interception, a mission-agnostic patch grammar with anchor-port references, a context-indexed fleet library, bandit-based selection with an explicit LLM-call arm, and replay-based admission with CMA-ES tuning (Sec. IV).
2) An implementation on Project AirSim with a schema-constrained ADK agent pipeline, a lockstep tick loop at $10\times$ real time whose decision traces are an exact function of the observation sequence, and a failure-injection curriculum calibrated against the simulator's measured response (Sec. V).
3) An evaluation against hand-coded, regenerate-on-failure, and five ablated baselines over a frozen phase in which all learning is disabled, with an exact decision-determinism test, and an analysis of what the admission pipeline caught and what it wrongly rejected (Secs. VI, VII).

II. Related Work

A. Classical Control Architectures

FSMs require transitions from every state to handle global events, so transition count grows quadratically with state count [1]. BTs [3], [4] standardize the node interface to a ternary status and decouple nodes; the survey by Iovino et al. [2] documents the gains in modularity and reactivity and the persistent limitation that BT structure is fixed at design time. Learned BTs via genetic programming [5], [6] or learned leaf policies [7] learn from reward alone and are sample-hungry when the needed structural change is semantically obvious.

B. Foundation Models for Control Synthesis

Code as Policies [9] prompts an LLM with a robot API to write programs; SayCan [10] and Inner Monologue [11] ground plans in affordances and feedback; VoxPoser [12] composes LLM reasoning and VLM grounding into value maps; VLA models [13], [14] map pixels to actions. Closest to us are BTGenBot [15], which fine-tunes small LLMs to emit BehaviorTree.CPP XML and validates it syntactically and in simulation, and LLM-BT [16]. These regenerate structure on demand and do not retain, verify against an empirical margin, or learn when to stop calling the model. Surveys [17], [18] discuss the latency, cost, and hallucination risks that motivate our design.

C. Combining LLMs with Learning

Eureka [19] has an LLM write reward functions that RL optimizes; RL-GPT [20] lets an LLM decide what to code and what to learn; Voyager [21] keeps a growing library of LLM-written skills retrieved by embedding. SCBT shares the library with Voyager and the LLM/optimizer division of labor with Eureka, but the stored artifacts are BT subtrees, admission is gated by simulated invariants and an empirical margin, and the stated end state is a system with no model running.

D. Multi-Robot Coordination and Verification

Classical MRTA [22]–[25] requires pre-defined tasks with comparable costs. LLM coordination [26], [27] describes robots by textual capability profiles; we adopt that idea as a *capability manifest* that gates which leaves a patch may use and declares substitutions across vehicle classes. Prior verification of generated control checks syntax and simulated execution [15]; we add invariant monitors, a landing-reserve constraint, and a required margin over the unpatched tree.

III. Problem Formulation

A. Behavior Trees

A BT is a rooted tree whose internal nodes are control-flow nodes (Sequence, Fallback, Parallel, decorators) and whose leaves are actions and conditions [1]. It is ticked at a fixed rate; each node returns Success, Failure, or Running. We use py_trees [8] with an XML dialect that mirrors BehaviorTree.CPP: Sequence/Fallback with memory, Reactive variants that re-evaluate all children every tick, Parallel, Inverter, ForceSuccess/Failure, Retry, Repeat. Leaves declare typed *ports*; numeric ports may carry bounds, and bounded ports are *tunable*. We write θ for a vector of tunable port values.

B. Vehicles and Capability Manifests

Each vehicle has a class m with a manifest C_m: the leaf types the class may execute, with port signatures and one-line descriptions, plus *substitutions* (e.g. a camera-less class substitutes `RequestScout` for `InspectRegion`). Classes differ physically (frame mass, speed cap enforced by the leaves, battery drain).

C. Missions and Success

A mission plan $(T_m,\ G_{\mathrm{req}},\ G_{\mathrm{opt}})$ is a base tree plus required and optional *goals*; goal leaves record completion on the blackboard. A mission *succeeds* if all required goals are reached and the vehicle lands. Mission *value* weights required goals 1 and optional goals 0.5. An abort to the hand-coded safe branch after the required goals are met is not a failure; an abort before them is. This definition matters: a recovery that merely returns to launch scores zero when a required goal is outstanding, which is what makes the battery curriculum (Sec. VI-A) a prioritization problem rather than a trivial one.

D. Failure Events

Definition 1 (Failure event): A failure event on a vehicle is $e = (v_f, c)$ where v_f is the deepest leaf that returned Failure in the tick in which the mission-critical subtree failed, and $c = (s, b, m, \tau, X)$ with s a textual context, b the telemetry snapshot, m the class, τ the last k status changes in the enclosing subtree, and X the XML of the enclosing subtree with v_f marked.

In the headless setting s is generated from telemetry (failed node type, leaf feedback, altitude, speed, wind, battery, GPS and link status); on a rendering host it is a VLM description. The vehicle class is deliberately excluded from s so that retrieval surfaces cross-class candidates.

E. Objective

Over a sequence of failure events the learning phase maximizes

$$\sum_t \big(y_t - \kappa\, u_t - \lambda\, \Delta t_t/100 - \mu\, \Delta E_t/10\big), \qquad (1)$$

where y_t is mission success after event t, u_t indicates an LLM call, Δt_t (s) and ΔE_t (% battery) are the time and energy from the event to the end of the episode, and κ, λ, μ weight cost, time and energy. The deployment objective is that the crystallized tree matches learning-phase success with no LLM, optimizer, or bandit running.

IV. The SCBT Framework

Fig. 1 shows the components and the boundary between what is deployed and what only runs during learning.

A. Failure Interception

A `MissionCritical` decorator wraps the mission subtree. When its child returns Failure and patching is enabled, it returns Running for that tick and raises a flag; the runner pauses the simulator, constructs e, and consults the Patch Manager. If a patch is applied, the mission subtree restarts on the next tick (leaves are idempotent: takeoff when airborne and navigation to a reached waypoint return Success immediately). If no patch is available, the decorator returns Failure on the next tick *without ticking its child*, and the root Fallback proceeds to the hand-coded abort branch. With patching disabled (frozen phase, baselines) the decorator is a pure pass-through. Each episode has a budget of $P_{\max}$ patches and $F_{\max}$ failures of the same node; already-applied patches are excluded from fallback selection. Without these limits the manager re-applied the same patch on the same anchor until the tree exceeded the interpreter's recursion limit.

B. Patch Grammar

Definition 2 (Patch): $\pi = (a, o, \sigma, \Theta_\sigma, r)$: anchor node name a, operator $o \in \mathcal{O} = \{$WrapFallback, GuardWith, WrapRetry, InsertBefore, InsertAfter, ReplaceSubtree$\}$, subtree σ whose leaves are in C_m (or have substitutions), tunable box Θ_σ with per-port bounds and initial values, and a rationale r.

WrapFallback replaces a by $\mathrm{Fallback}(a, \sigma)$; GuardWith by $\mathrm{Sequence}(\sigma, a)$; ReplaceSubtree is the only operator that deletes logic and requires a rationale. Two features keep patches *mission-agnostic*: an attribute value `{@p}` refers to port p of the anchor leaf and is resolved when the patch is applied, and a `GoToRequiredGoal` leaf flies to the next outstanding required goal read from the blackboard. Literal coordinates copied from the event are therefore unnecessary; during development, a patch that contained them was reused on a different mission and flew to the wrong place, which is why the rule exists. Wrapper nodes created by a patch are named after the patch identifier and nodes in σ whose names collide with existing ones are renamed, so anchors remain unique and a crystallized tree is readable.

C. LLM Proposer

The proposer is a Google ADK [31] `SequentialAgent` of two `LlmAgents` with Pydantic output schemas: a *ContextAnalyst* producing a diagnosis (summary, likely cause, strategy, anchor) and a *PatchProposer* producing exactly one patch. Malformed output is rejected by the schema before our checker runs. The message contains the rendered event, the mission's goal structure, the success criterion and the hard invariants, the manifest, the grammar, and up to three nearest library entries *with their outcome records* (admission status, uses, successes, rejection reason). A *repair loop* re-prompts with the static checker's message up to R times. The success criterion was added after early runs showed the model proposing a safe return-to-launch that abandoned the required goal; stating the objective explicitly is a prompt-engineering fix, but the pipeline's protection against such patches is admission, not prompting.

D. Embedding, Library, and Bandit

$\phi(c)$ concatenates a MiniLM sentence embedding [30] of s, one-hot encodings of the failed node type and the class, and normalized telemetry. Retrieval uses cosine similarity on the text and node-type blocks only (class excluded) with threshold ρ. The library stores, per patch, the embedding, XML, tuned θ per class, admission status and reason, provenance (event, rationale, proposer, attempts, admission record), and use/success counts.

Selection is linear Thompson sampling [29] over arms $\{\text{retrieved patches}\} \cup \{$CallLLM$\}$ with context $[\phi(c), 1]$; new arms get an optimistic prior mean, CallLLM a neutral one. Rewards per (1) are computed at episode end and credited to the arm played; when CallLLM produced a patch, that patch is additionally credited with the reward plus κ. As stored arms accumulate evidence, CallLLM, which always pays κ, is selected less often; LLM usage decays because it is learned, not scheduled. The frozen phase replaces sampling by the posterior-mean argmax.

E. Replay-Based Admission and Tuning

A *replay* re-runs the originating episode (same mission, same failure type) on a dedicated simulator instance with a candidate tree compiled from the *live tree as it currently stands* plus the patch, with patching disabled. Replaying from mission start, rather than respawning at the recorded state, keeps controller and sensor state consistent and costs one episode per replay (5–15 s wall at $10\times$; four runtime instances serve replays in parallel). The event neighbourhood $\mathcal{N}(e)$ perturbs injection parameters by $\pm p$ and the trigger time by ± 2 s, *clipped to the failure's catalogue range*; without clipping, a perturbed battery level drifted above the guard and the replay never failed.

Tuning. For $\dim \Theta_\sigma > 0$, CMA-ES [28] in the unit box with budget B maximizes

$$J(\theta) = \hat{p}_{\mathrm{succ}} - \lambda\, \bar{t}/100 - \mu\, \bar{E}/10 - \nu\, \mathbf{1}[\text{unsafe}], \qquad (2)$$

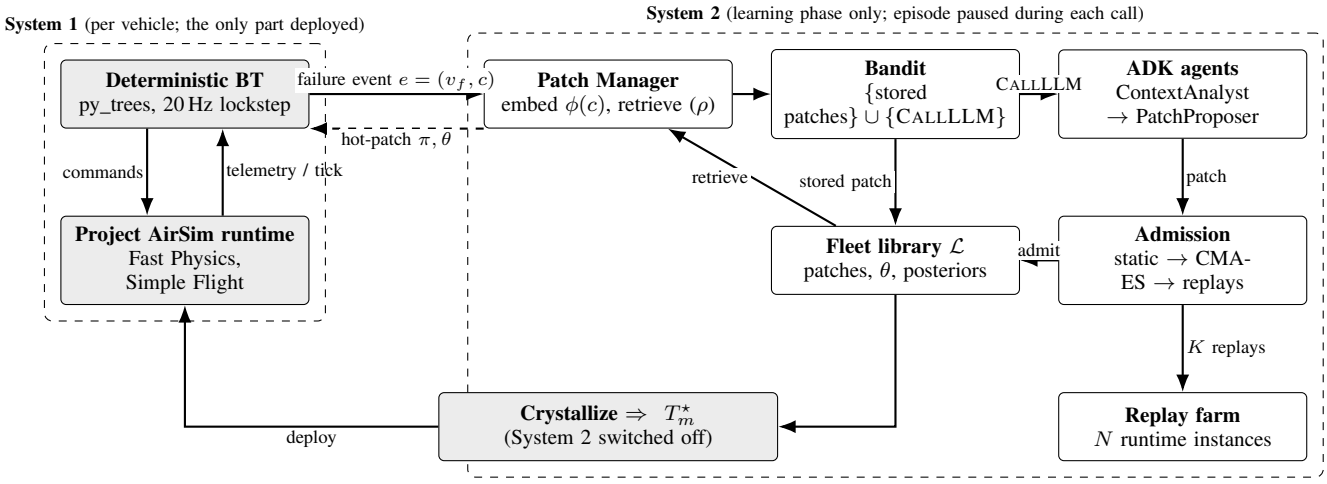


Fig. 1. SCBT architecture. The vehicle's tick path contains only a deterministic BT. The Patch Manager runs above it during the learning phase, with the live episode paused during each call, and is absent after crystallization.

where *unsafe* means any invariant violation or landing reserve below 10%. The safety term is in the objective because without it CMA-ES traded reserve for success rate and produced a patch that landed at 9.8%.

Stages. (1) *Static*: operator and anchor exist; leaves in C_m or substitutable; ports exist, parse, and lie within bounds; placeholders declared and used; anchor references resolvable; depth limit; REPLACESUBTREE has a rationale and does not remove `MissionCritical`. (2) *Constraint*: K replays of the tuned patch; per-tick monitors for geofence, altitude, speed (class cap with margin), inter-vehicle separation; reject on any violation tick or landing reserve $< 10\%$. (3) *Empirical*: success rate $\geq \rho_{\min}$ and $\geq$ the rate of the current tree without the candidate (same perturbation seeds) $+\delta$. The baseline is the *current* tree, so a candidate for a failure the tree already handles is rejected for lack of margin; this was the most common stage-3 rejection in our runs. Admission errors reject rather than abort the run.

F. Cross-Class Transfer

A patch is a candidate for class m' if every leaf is in $C_{m'}$ or has a substitution, which the static checker applies in place; θ is re-tuned from the originating class's optimum and the patch is re-admitted for m'. Retrieval is class-agnostic, so such candidates are visible.

G. Crystallization

When the windowed LLM-call rate falls below ϵ and success is at least γ, each class's admitted patches are applied to T_m in admission order with $\theta^{(m)}$ fixed; patches sharing an anchor become children of one Fallback ordered by bandit posterior mean. The result $T_m^\star$ runs with no LLM, optimizer, or bandit. Under the idealized assumptions of clustered contexts and a positive per-call admission probability, expected LLM calls are bounded by $Q|\mathcal{M}|/p$ plus bandit exploration; we state this only to make explicit what the empirical amortization curve measures (how far real contexts behave like a finite set of clusters at the chosen ρ).

V. IMPLEMENTATION

Simulator. Microsoft Project AirSim [33] (successor of AirSim [32]), headless runtime: Fast Physics, Simple Flight controller, IMU/GPS/barometer/battery sensors, no rendering and no mesh collisions. Scene and robot configurations are generated programmatically per class (frame mass, battery capacity and drain). The steppable clock is driven in *lockstep*: the simulator is paused and advanced by exactly one tick period (50 ms) per BT tick at $10\times$ real time. Two client-side races were removed so that the tick trace is a function of the observation sequence alone: commands issued during a tick are flushed, and a synchronous barrier request is issued, before the step; and action leaves decide completion from the per-tick telemetry snapshot rather than from the server's reply, whose arrival time is wall-clock dependent. The simulator itself is not bit-reproducible: its command-execution thread is wall-clock driven, and with no BT in the loop five identical lockstep runs produced three trajectories differing at the 10^{-4} m level. End-to-end repetitions can therefore still differ by a threshold crossing; Sec. VII reports both the end-to-end and the exact offline measurement. Several runtime instances on distinct port pairs serve the live episode and the replay farm.

BT runtime. py_trees [8] with 17 leaf types over the Project AirSim Python client: `Takeoff`, `Land`, `ReturnToLaunch`, `GoToNED`, `GoToRequiredGoal`, `MoveRelative`, `Hover`, `WaitSimTime`, `SetSpeed`, `InspectRegion`/`RequestScout`, and conditions `IsAirborne`, `IsBatteryOk`, `IsWindOk`, `IsGpsOk`, `IsLinkOk`, `IsInsideGeofence`. Actions start an asynchronous simulator command on first tick and poll it; telemetry is refreshed once per tick into a dictionary that leaves read, so the tick trace is a function of the telemetry sequence. Blackboards are namespaced per (simulator

```
Fallback root
  MissionCritical mission
    Sequence mission_seq
      Takeoff
      SetSpeed
      ReactiveSequence guarded_flight
        IsBatteryOk min_pct=25
        IsInsideGeofence
        Sequence waypoints
          GoToNED wp1 ... GoToNED wp_n (goal_id)
      Land
  Sequence abort
    ReturnToLaunch
    Land
```

Fig. 2. Base tree T_m. Guards are re-evaluated every tick; a failure anywhere under `MissionCritical` raises a failure event, and if no patch is applied the hand-coded `abort` branch runs.

instance, vehicle); the replay farm runs vehicles with the same names, and without the instance prefix the live tree commanded the farm's drone.

Base trees. Fig. 2 shows the base tree, identical for all classes up to waypoints and limits. Missions: M1 *transport* (deliver 40 m north, required; return; land) and M2 *survey* (three lanes of 30 m; lane 1 required, lanes 2–3 optional; return; land). A nominal M1 uses 12.7% battery and M2 20.7%.

LLM and embedding. Gemini 3.1 Pro (`gemini-3.1-pro-preview`) at temperature 0.2 through ADK; mean latency per structured call 31–105 s depending on method (two calls per proposal). Sentence embedding `all-MiniLM-L6-v2`. A mock proposer that replays hand-written patches lets the full System-2 pipeline run offline for tests.

Parameters. $\rho = 0.80$, $\kappa = 0.30$, $\lambda = \mu = 0.10$, $\nu = 1$, $R = 2$, $P_{\max} = 3$, $F_{\max} = 2$, $K = 3$, $B = 4$, $p = 0.20$, $\rho_{\min} = 0.75$, $\delta = 0.25$.

VI. Experimental Design

A. Failure Curriculum

Failures are injected at a random time in $[6, 14]$ s. Headless-capable: **F1** wind gust; **F2** GPS noise then loss; **F5** landing pad shifted; **F6** no-fly zone (geofence shrinks across the route); **F7** battery capacity reduced; **F8** link dropout; **F12** separation conflict (two vehicles). Obstacles, pad occupancy, camera dropout, and target search (F3, F4, F9–F11) require the rendering host and are future work. Two calibrations matter. *F1*: the Fast Physics quadrotor rejects uniform wind up to 20 m/s almost completely (< 1 m drift in 8 s) and saturates above $\sim$25 m/s (47 m drift in 8 s at 30 m/s; a 40 m leg under 25 m/s ended 226 m from its target); F1 therefore samples 22–28 m/s for 15–30 s, which is "beyond tracking capability" by measurement. *F7*: battery set to 17–24% (below the 25% guard) with drain $\times$1.3–2.2, so that completing the required leg under a reduced reserve is feasible in most but not all instances. Held-out variants use disjoint parameter sub-ranges.

B. Baselines and Ablations

B0 hand-coded expert tree with recovery branches for all failures; **B1** whole-tree regeneration by the LLM on every failure, no library; **B2** patch grammar and admission, no library (every failure calls the LLM); **B3** no CMA-ES (LLM's initial values); **B4** no bandit (nearest neighbour, else LLM); **B5** static admission only; **B6** per-class libraries; **SCBT** full. B2–B6 are configuration flags of the same code path.

C. Protocol and Metrics

Learning phase: 20 episodes, mission $\sim U\{\text{M1, M2}\}$, failure $\sim U\{\text{F1, F7}\}$, shared seeds across methods; SCBT over seeds 0–2, ablations over seed 0. *Frozen phase*: crystallize; evaluate $T_m^\star$ on 20 curriculum and 20 held-out episodes with all learning disabled (no Patch Manager, farm, LLM, or bandit object is instantiated). B1 has no frozen artifact. *Determinism*: 10 end-to-end repetitions of the first scenario (distinct trace hashes, tick spread), and for each repetition three offline re-ticks of the frozen tree against that repetition's recorded telemetry log with no-op commands (distinct hashes; whether the offline hash equals the online one). Metrics: success rate and mission value per failure type; abort rate; LLM calls per episode and cumulative (the *amortization curve*); library hit rate (same/cross class); admission rejection rate per stage; recovery latency decomposed into LLM, tuning, and admission; bandit regret; parameter-transfer gap; crystallized-tree size versus B0; held-out generalization.

VII. Results

A. Learning Phase

Table I and Fig. 3 summarize the learning phase. SCBT used 11 ± 1.4 LLM calls over 20 episodes and 0.33 calls per episode in the second half; whole-tree regeneration (B1) used 26 and patching without a library (B2) 22, both still above one call per episode at the end. The library is what produces amortization: B2 differs from SCBT only in not storing patches. The bandit's contribution is visible in B4, which calls the LLM whenever retrieval misses (18 calls, 0.70 per episode in the second half) but reaches the highest single-seed learning-phase success (0.80) by trying stored patches more often (25 hits). Across SCBT seeds, 12–14 of 23–24 failure events were served from the library.

Admission was selective: 2–3 proposals admitted and 7–9 rejected per seed, by stage 2 (landing reserve 0.6–8.0%, or 238–1009 geofence-violation ticks under wind) and stage 3 (no margin over the current tree, nominal regression, or success below $\rho_{\min}$). Four rejections were admission *errors* (malformed XML from the model reaching the compiler), which the pipeline treats as rejections. B5, which skips replays, admitted all 10 of its proposals. Per SCBT seed, admission consumed 103–203 replay episodes.

B. Frozen Phase

Table II reports the crystallized trees evaluated with every learning component disabled. SCBT's trees, with 28 nodes for M1 and 33 for M2 and two patches each, reached $97 \pm 2\%$ success and 0.78 ± 0.04 mission value on the curriculum, against 12% and 0.12 for the unpatched base tree and 98%/0.83 for the hand-coded expert tree B0. Per failure type, SCBT recovered

TABLE I

LEARNING PHASE, 20 EPISODES, GEMINI 3.1 PRO. SCBT: MEAN±SD OVER SEEDS 0–2; ABLATIONS: SEED 0. "CALLS, EP. 11–20" IS LLM CALLS PER EPISODE OVER THE SECOND HALF. "HITS" ARE FAILURE EVENTS SERVED FROM THE LIBRARY.

Method	Seeds	Success	Value	LLM calls	Calls/ep. 11–20	Adm./rej.	Hits	Replays
B1 regenerate whole tree	1	0.55	0.50	26	1.20	11 / 11	0	308
B2 no library	1	0.40	0.33	22	1.00	9 / 11	0	342
B3 no CMA-ES tuning	1	0.65	0.57	12	0.10	3 / 9	10	120
B4 no bandit (nearest nb.)	1	0.80	0.65	18	0.70	4 / 10	25	252
B5 static admission only	1	0.85	0.70	10	0.40	10 / 0	22	0
B6 no cross-class transfer	1	0.65	0.55	13	0.40	2 / 9	13	188
SCBT (full)	3	0.75 ± 0.14	0.62 ± 0.11	11 ± 1.4	0.33	2.3 / 8.0	12.3	103–203

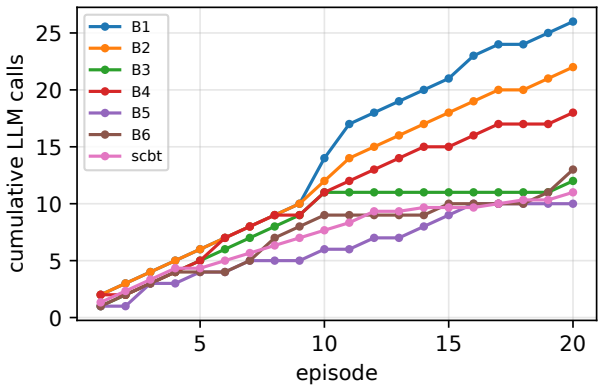


Fig. 3. Cumulative LLM calls versus episode (SCBT: mean over three seeds; others: seed 0). Methods without a library (B1, B2) remain near one call per episode; SCBT, B3, B5 and B6 flatten.

22/24 F1 and 36/36 F7 episodes (B0: 23/24 and 36/36). A tree no human wrote matches, on the distribution it learned from, one written by an engineer with the failure catalogue in hand, with no model in the loop.

On held-out variants every method degrades: F1 held-out gusts (28–30 m/s for 30–36 s) push the vehicle outside the geofence in 14 of 20 episodes regardless of the tree, and B0 recovers only 4/36 of them. SCBT (4/36 F1, 22/24 F7) is statistically indistinguishable from B0 there. The exception is B5, discussed below.

C. Determinism

In all eleven frozen cells, re-ticking the crystallized tree offline against a recorded telemetry log produced a single trace hash over three re-ticks, and that hash equaled the hash of the online run that produced the log (60 of 60 logs across all cells). The control logic is therefore a pure function of its observations, as claimed. End to end, seven cells gave one hash in ten repetitions; the remaining four (B3, B6, two SCBT seeds) gave 2–4 distinct hashes with tick spreads of 1–33 ticks (0.05–1.65 s) and the same final outcomes. We traced the residual to the simulator: its command-execution thread is scheduled by wall-clock time, so lockstep stepping is exact (a 50 ms request always advances 17 physics steps) but the moment a command takes effect within a step is not, and trajectories drift at the 10^{-4} m level. Threshold crossings (arrival tolerance, guard conditions) occasionally flip by one tick. We report this rather than hide it, since a reader deploying crystallized trees on hardware will face the same distinction: the tree is deterministic, the world is not.

D. What Admission Caught, and What It Got Wrong

Three observations argue for empirical admission; a fourth argues against the particular stage-2 rule we used.

1) **Safe but useless patches.** With static-only admission, the model's first battery recovery was `ReturnToLaunch` alone: safe, admitted, and 0 for 6 on mission success because it abandoned the required goal. The bandit then correctly concluded the LLM was not worth κ and converged on the bad patch. Stage 3 rejects such patches for lack of margin.
2) **Optimizer exploiting the objective.** CMA-ES without the safety term chose a 13.2% reserve that landed at 9.8%; adding the invariant penalty to (2) yielded 13.9% and 12.5% at landing. Across the matrix, stage 2 rejected five battery patches that would have landed at 0.6–8.0%.
3) **Nominal regression.** Before the nominal-replay check, a REPLACESUBTREE that deleted the optional survey lanes was admitted, used seven times with seven successes, and crystallized: it never failed a required goal because it never attempted an optional one. The check now rejects any drop in nominal mission value.
4) **Blaming the patch for the wind.** Stage 2 rejects a patch if *any* replay violates an invariant, without reference to what the unpatched tree does under the same perturbation. Under F1 the gust itself carries the vehicle across the geofence; replays of a wind-recovery patch therefore logged 238–1009 violation ticks and the patch was rejected, although the violation was not of its making. SCBT consequently crystallized with few wind recoveries, whereas B5, which admits every statically valid patch, kept eight WRAPFALLBACK and two WRAPRETRY patches (84/89 nodes) and recovered 14/14 held-out F1 episodes that no other method, including the expert tree, could. B5's frozen trees were not unsafe on this curriculum (no invariant violations in 20 curriculum episodes; three landings below 10% reserve, versus two for SCBT). The correct stage-2 rule is *relative*: reject violations the patch introduces beyond

TABLE II

FROZEN PHASE: CRYSTALLIZED TREES WITH ALL LEARNING DISABLED, 20 CURRICULUM AND 20 HELD-OUT EPISODES PER SEED (B0, BASE, SCBT: SEEDS 0–2; ABLATIONS: SEED 0). "END-TO-END": DISTINCT TRACE HASHES OVER 10 REPETITIONS OF ONE SCENARIO (WORST SEED) AND MAXIMUM TICK SPREAD. "OFFLINE": DISTINCT HASHES OVER 3 RE-TICKS OF THE FROZEN TREE AGAINST EACH REPETITION'S RECORDED TELEMETRY, AND THE NUMBER OF LOGS WHOSE OFFLINE HASH EQUALS THE ONLINE ONE. NODES: M1/M2 TREES (PATCHES APPLIED).

	Curriculum		Held-out		End-to-end		Offline	Nodes
Method	Succ.	Value	Succ.	Value	Distinct	Spread	Dist./faithful	M1 / M2
Base (unpatched)	$.12 \pm .09$	$.12 \pm .09$	0.00	0.00	–	–	–	15 / 20 (0)
B0 expert	$.98 \pm .02$	$.83 \pm .06$	$.47 \pm .06$	$.37 \pm .08$	1 / 10	0	1/3, 30/30	–
B2 no library	1.00	0.85	0.40	0.30	1 / 10	0	1/3, 10/10	77 / 82 (9+9)
B3 no CMA-ES tuning	1.00	0.85	0.45	0.35	4 / 10	33	1/3, 10/10	33 / 38 (3+3)
B4 no bandit	0.80	0.62	0.15	0.10	1 / 10	0	1/3, 10/10	37 / 42 (4+4)
B5 static adm. only	0.95	0.80	**1.00**	**0.75**	1 / 10	0	1/3, 10/10	84 / 89 (10+10)
B6 no transfer	1.00	0.82	0.45	0.30	2 / 10	30	1/3, 10/10	33 / 38 (2+2)
SCBT (full)	$\mathbf{.97 \pm .02}$	$.78 \pm .04$	$.43 \pm .10$	$.32 \pm .10$	3 / 10	12	1/3, 30/30	28 / 33 (2+2)

those of the unpatched tree under the same seeds, as stage 3 already does for success. We leave this as the first change for future work rather than re-run with it, so that the reported numbers reflect the pipeline as specified in Sec. IV-E.

VIII. DISCUSSION

What crystallization buys. The frozen artifact is deterministic and inspectable, and provenance ties every subtree to the failure that produced it, the model's rationale, and the admission record. It does not generalize beyond what the learning phase encountered; the held-out metric exists to make that limit visible.

Patches versus regeneration. Whole-tree regeneration (B1) discards verified logic, makes differences between episodes unattributable, and in our runs used the most LLM calls and the most replays (308) while achieving the lowest learning-phase success among library methods. Patches preserve attribution and compose; the crystallized SCBT tree is the base tree plus two readable edits.

Admission is a dial, not a filter. The B5 result is the most informative in the matrix: on this curriculum, admitting every statically valid patch produced larger trees that generalized better, and the safety cost was small. Our stage-2 rule was too strict in one specific, correctable way (Sec. VII-D); but the general point stands that admission trades coverage for guarantees, and the right setting depends on how much the operator trusts the simulator's invariants versus the model's proposals. The framework makes that trade explicit and measurable; it does not make it for you.

Safety. Admission is necessary, not sufficient. Randomized replay around a recorded event does not cover the deployment distribution; the invariant monitors are the operative safety mechanism and run identically in replay and in the live episode. We make no formal-verification claim.

Limitations. Two failure types; single vehicle per episode (the fleet, F12 and cross-class transfer are implemented but not exercised, so B6 differs from SCBT only by configuration); ablations on one seed; headless host, so context is telemetry-derived and the perception-dependent failures are absent; Simple Flight rather than PX4. The held-out F1 band is largely infeasible for every method and therefore measures robustness to an unrecoverable condition rather than generalization; a held-out band inside the recoverable regime would be more informative. Four admission errors were caused by malformed model output reaching the compiler and should be caught at the static stage.

IX. CONCLUSION

The adaptability of foundation models and the determinism of classical control are usually presented as a choice. SCBT sequences them: an LLM proposes structure when a Behavior Tree fails, CMA-ES tunes it, replay admission gates it, a bandit learns when to stop asking, and the library compiles into an ordinary deterministic tree. On Project AirSim, the learning phase needed 11 ± 1.4 LLM calls in 20 episodes against 26 for regeneration, and the crystallized trees, with two patches each, matched a hand-coded expert tree on the training distribution with no model running and with decision traces that are an exact function of their observations. The admission pipeline caught safe-but-useless patches, an optimizer exploiting its objective, and a patch that deleted mission goals; it also wrongly rejected wind recoveries by attributing the wind's geofence excursions to the patch, and the static-only ablation generalized better as a result. Future work: relative invariant checks in admission, the fleet and cross-class experiments, the rendering host with VLM context, and deployment of crystallized trees on PX4 hardware.

REFERENCES

[1] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018. arXiv:1709.00084.

[2] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of behavior trees in robotics and AI," *Robotics and Autonomous Systems*, vol. 154, p. 104096, 2022.

[3] P. Ögren, "Increasing modularity of UAV control systems using computer game behavior trees," in *Proc. AIAA GNC Conf.*, 2012.

[4] A. Marzinotto, M. Colledanchise, C. Smith, and P. Ögren, "Towards a unified behavior trees framework for robot control," in *Proc. ICRA*, 2014, pp. 5420–5427.

[5] M. Colledanchise, R. Parasuraman, and P. Ögren, "Learning of behavior trees for autonomous agents," *IEEE Trans. Games*, vol. 11, no. 2, pp. 183–189, 2019.

[6] M. Iovino, J. Styrud, P. Falco, and C. Smith, "Learning behavior trees with genetic programming in unpredictable environments," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2021, pp. 4591–4597.

[7] R. de P. Pereira and P. M. Engel, "A framework for constrained and adaptive behavior-based agents," arXiv:1506.02312, 2015.

[8] D. Stonier et al., "py_trees," software, v2.6.0, https://github.com/splintered-reality/py_trees. Accessed: Oct. 18, 2026.

[9] J. Liang et al., "Code as Policies: Language model programs for embodied control," in *Proc. ICRA*, 2023. arXiv:2209.07753.

[10] M. Ahn et al., "Do As I Can, Not As I Say: Grounding language in robotic affordances," in *Proc. CoRL*, 2022. arXiv:2204.01691.

[11] W. Huang et al., "Inner Monologue: Embodied reasoning through planning with language models," in *Proc. CoRL*, 2022. arXiv:2207.05608.

[12] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D value maps for robotic manipulation with language models," in *Proc. CoRL*, 2023. arXiv:2307.05973.

[13] A. Brohan et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," arXiv:2307.15818, 2023.

[14] M. J. Kim et al., "OpenVLA: An open-source vision-language-action model," arXiv:2406.09246, 2024.

[15] R. A. Izzo, G. Bardaro, and M. Matteucci, "BTGenBot: Behavior tree generation for robotic tasks with lightweight LLMs," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2024, doi:10.1109/IROS58592.2024.10802304.

[16] H. Zhou, Y. Lin, L. Yan, J. Zhu, and H. Min, "LLM-BT: Performing robotic adaptive tasks based on large language models and behavior trees," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024, doi:10.1109/ICRA57147.2024.10610183.

[17] R. Firoozi et al., "Foundation models in robotics: Applications, challenges, and the future," *Int. J. Robotics Research*, 2024. arXiv:2312.07843.

[18] Z. Xu et al., "A survey on robotics with foundation models: Toward embodied AI," arXiv:2402.02385, 2024.

[19] Y. J. Ma et al., "Eureka: Human-level reward design via coding large language models," in *Proc. ICLR*, 2024. arXiv:2310.12931.

[20] S. Liu, H. Yuan, M. Hu, Y. Li, Y. Chen, S. Liu, Z. Lu, and J. Jia, "RL-GPT: Integrating reinforcement learning and code-as-policy," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[21] G. Wang et al., "Voyager: An open-ended embodied agent with large language models," *Trans. Machine Learning Research*, 2024. arXiv:2305.16291.

[22] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *Int. J. Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[23] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955.

[24] B. P. Gerkey and M. J. Matarić, "Sold!: Auction methods for multirobot coordination," *IEEE Trans. Robotics and Automation*, vol. 18, no. 5, pp. 758–768, 2002.

[25] E. Nunes and M. Gini, "Multi-robot auctions for allocation of tasks with temporal constraints," in *Proc. AAAI*, 2015, pp. 2110–2116.

[26] Z. Mandi, S. Jain, and S. Song, "RoCo: Dialectic multi-robot collaboration with large language models," in *Proc. ICRA*, 2024. arXiv:2307.04738.

[27] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *Proc. ICRA*, 2024. arXiv:2309.15943.

[28] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

[29] S. Agrawal and N. Goyal, "Thompson sampling for contextual bandits with linear payoffs," in *Proc. ICML*, 2013, pp. 127–135.

[30] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[31] Google, "Agent Development Kit (ADK) for Python," software, v2.11.0, https://github.com/google/adk-python. Accessed: Oct. 18, 2026.

[32] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, 2017. arXiv:1705.05065.

[33] IAMAI Consulting Corp. and Microsoft, "Project AirSim," open-source software, v1.1.0, https://github.com/iamaisim/ProjectAirSim. Accessed: Oct. 18, 2026.